

Payment Gateway Integration

Business Requirements Document v1.0

Confidential | FinServe India Pvt. Ltd.

1. Project Overview

This document outlines the requirements for integrating RazorPay Payment Gateway API into the QuickLend lending platform to enable automated loan disbursement and EMI collection. The integration must support UPI, NEFT, IMPS, and card-based payment methods for both B2C disbursements and recurring EMI collections.

2. API Endpoints Required

POST /v1/payments/create - Initiate a new payment (disbursement or collection)

GET /v1/payments/{payment_id} - Fetch payment status by ID

POST /v1/payments/{payment_id}/refund - Initiate full or partial refund

GET /v1/settlements - List settlement details for a date range

POST /v1/mandates/create - Create recurring payment mandate (e-NACH / UPI autopay)

GET /v1/mandates/{mandate_id} - Fetch mandate status

POST /v1/mandates/{mandate_id}/cancel - Cancel an active mandate

3. Field Mapping Requirements

The following fields must be mapped from the loan management system to the payment API:

- borrower_name -> customer_name (mandatory)
- borrower_pan -> pan (mandatory)
- borrower_phone -> contact (mandatory)
- borrower_email -> email (optional)
- loan_account_number -> reference_id (mandatory)
- disbursement_amount -> amount (mandatory, in paise)
- bank_account_number -> account_number (mandatory)
- bank_ifsc -> ifsc_code (mandatory)
- payment_method -> method (mandatory: upi/neft/imps/card)
- emi_amount -> recurring_amount (mandatory for mandates)
- emi_frequency -> frequency (monthly/quarterly)
- mandate_start_date -> start_at (mandatory for mandates)
- mandate_end_date -> expire_by (mandatory for mandates)

4. Authentication Requirements

- API Key + Secret based authentication (HTTP Basic Auth)
- Key ID sent as username, Key Secret as password
- Webhook signature verification using HMAC-SHA256
- All credentials must be stored in encrypted vault
- Separate keys for test and production environments
- IP whitelisting required for production access

5. Response Handling

The payment gateway will return:

- payment_id: Unique payment identifier (e.g., pay_LnR5hmG3YVzB)
- order_id: Order reference ID
- status: created / authorized / captured / refunded / failed
- amount: Amount in paise (integer)
- currency: INR
- method: upi / card / netbanking / wallet / bank_transfer
- upi_transaction_id: UPI transaction reference (for UPI payments)
- bank_reference: Bank UTR number (for NEFT/IMPS)
- error_code: Error code if payment failed
- error_description: Human-readable error message
- refund_id: Refund identifier (for refund responses)
- settlement_id: Settlement batch reference

6. Webhook Events

The gateway sends real-time webhook notifications for:

- payment.authorized : Payment authorized, pending capture
- payment.captured : Payment successfully captured
- payment.failed : Payment attempt failed
- refund.processed : Refund completed successfully
- settlement.processed : Daily settlement processed
- mandate.activated : Recurring mandate activated
- mandate.paused : Mandate paused by customer
- mandate.cancelled : Mandate cancelled

Each webhook payload must be verified using HMAC-SHA256 with the webhook secret.

7. Security Requirements

- All communication over TLS 1.2+
- PII data (PAN, Aadhaar, bank account) must be masked in logs
- PCI-DSS Level 1 compliance for card data handling
- Audit trail for all payment transactions
- Data retention: 7 years for financial records as per RBI guidelines
- Access control: role-based, only finance team can trigger refunds
- Webhook endpoint must validate source IP range

8. SLA Requirements

- API response time: 5000ms maximum
- Payment processing: Real-time for UPI, T+1 for NEFT
- Settlement cycle: T+2 business days
- API availability: 99.95% uptime

- Rate limit: 50 requests per second
- Timeout: 30 seconds for payment initiation
- Webhook delivery: Within 5 minutes of event

9. Error Handling Requirements

- Retry on 5xx errors with exponential backoff (base 2s, max 60s)
- Maximum 5 retries for failed disbursements
- Idempotency key required for payment creation (to prevent duplicates)
- Alert on consecutive failures (> 3 in 5 minutes)
- Fallback: Queue failed payments for manual review
- Log all errors with correlation ID and bank reference
- Dead letter queue for permanently failed webhooks

10. Testing Requirements

- Unit tests for all field mapping transformations
- Integration tests with RazorPay sandbox environment
- End-to-end tests for full payment lifecycle
- Load testing: 500 concurrent payment requests
- Chaos testing: simulate gateway downtime and verify retry behavior
- Reconciliation tests: verify settlement amounts match